Smart Rockets using Genetic Algorithm

Dipankar Mazumder
Btech (CSE)
JIS University
Kolkata, India
dipankarmazumder9831@gmail.com

Krishnendu Pal
Btech (CSE)
JIS University
Kalna, India
palkrishnendu2@gmail.com

Dr. Saikat Maity
JIS University
Kolkata, India
saikat.maity@jisuniversity.ac.in

1 ABSTRACT

In this project based on the concepts of machine learning. Concepts like Natural Selection, Mutation, Genes Generation, Fitness Evaluation, Crossover etc are implemented in this project. This is a Evolution Simulator designed in p5.js. In simple words, a bunch of rockets (population) teach themselves how to reach from one point to another no matter how treacherous the path is, meaning, even if there are obstacles in the path, they learn to go around it, without human assistance. This the Rockets do through the Genetic Evolution over some successive generations.

2 KEYWORDS

Self Driving Algorithm, Genetic Algorithm, Selection, Crossover, Mutation, Mating Pool, p5.js, DNA, Genes, JavaScript, Fitness, Population, Evolution.

3 INTRODUCTION

Like other computational systems inspired by natural systems, genetic algorithms have been used in two ways: as techniques for solving technological problems, and as simplified scientific models that can answer questions about nature.

Automatic programming—i.e., having computer programs automatically write computer programs—has a long history in the field of artificial intelligence. This process keeps on iterating and at the end, a generation with the fittest individuals will be found. This notion can be applied for a search problem. We consider a set of solutions for a problem and select the set of best ones out of them.[ix][xiii][xvi]

Thus genetic algorithms implement the optimization strategies by simulating evolution of species through natural selection[i] [xii]

Six phases are considered in a genetic algorithm.

(i) Initial population
(ii) Fitness function
(iii) Selection
(iv) Crossover
(v) Mutation
(vi) Termination

Initial Population: The process begins with a set of individuals which is called a Population. Each individual is a solution to the problem you want to solve. An individual is characterized by a set of parameters (variables) known as Genes. Genes are joined into a string to form a Chromosome (solution). In a genetic algorithm, the set of genes of an individual is represented using a string, in terms of an alphabet. Usually, binary values are used (string of 1s and 0s). We say that we encode the genes in a chromosome.[xii][xiv]

Fitness Function: The fitness function determines how fit an individual is (the ability of an individual to compete with other individuals). It gives a fitness score to each individual. The probability



Figure 1. An array representation of initial population

that an individual will be selected for reproduction is based on its fitness score.[ii][xv]

Selection: Selection is the component which guides the algorithm to the solution by preferring individuals with high fitness over low-fitted ones. It can be a deterministic operation, but in most implementations it has random components.[iii][xx]

Crossover: In sexual reproduction, as it appears in the real world, the genetic material of the two parents is mixed when the gametes of the parents merge. Usually, chromosomes are randomly split and merged, with the consequence that some genes of a child come from one parent while others come from the other parents.[iv][xvii]

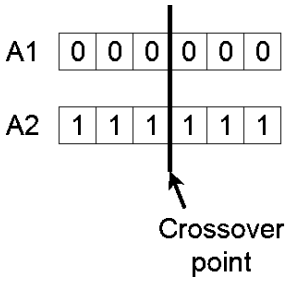


Figure 2: An array representation of crossover point

Offspring are created by exchanging the genes of parents among themselves until the crossover point is reached.

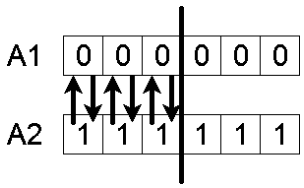


Figure 3: An array representation of Exchanging genes among parents

The new offspring are added to the population.

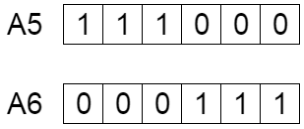


Figure 4: An array representation of the new offspring

Mutation: The last ingredient of our simple genetic algorithm is mutation—the random deformation of the genetic information of an individual by means of radioactive radiation or other environmental influences. In real reproduction, the probability that a certain gene is mutated is almost equal for all genes.[iv][xxi]

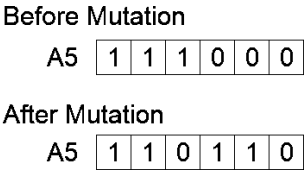


Figure 5: An array representation of before and after mutation

4 WORKING OF GENETICS AND CROSSOVERIN PROPOSED WORK

In the context of genetic algorithms, some biological terms are used in the spirit of analogy with real biology, though the entities they refer to are much simpler than the real biological ones. A chromosome can be conceptually divided into genes— each of which encodes a particular protein. Very roughly, one can think of a gene as encoding a trait, such as eye color. The different possible ”settings” for a trait (e.g., blue, brown, hazel) are called alleles. Each gene is located at a particular locus (position) on the chromosome. [v]

4.1 Insight about Genetics

Many organisms have multiple chromosomes in each cell. The complete collection of genetic material (all chromosomes taken together) is called the organism’s genome.

The genotype gives rise, under fetal and later development, to the organism’s phenotype—its physical and mental characteristics, such as eye color, height, brain size, and intelligence.[vi][xix]

4.2 Genetic Algorithm in living organism

All living organisms consist of cells, and each cell contains the same set of one or more chromosomes—strings of DNA—that serve as a ”blueprint” for the organism. A chromosome can be conceptually divided into genes— each of which encodes a particular protein.

In genetic algorithms, the term chromosome typically refers to a candidate solution to a problem, often encoded as a bit string. The ”genes” are either single bits or short blocks of adjacent bits that encode a particular element of the candidate solution (e.g., in the context of multiparameter function optimization the bits encoding a particular parameter might be considered to be a gene).

Most applications of genetic algorithms employ haploid individuals, particularly, singlechromosome individuals. The genotype of an individual in a GA using bit strings is simply the configuration of bits in that individual’s chromosome.

5 PROPOSED WORK AND PLATFORM

This project is entirely coded on the language JavaScript. It also uses the library called p5.js. This library is best suited for graphical applications like this.

JavaScript: JavaScript is a scripting or programming language that allows you to implement complex features on web pages — every time a web page does more than just sit there and display static information for you to look at — displaying timely content updates, interactive maps, animated 2D/3D graphics, scrolling video jukeboxes, etc.

p5.js: p5.js is for writing software to make images, animations, and interactions. The idea is to write a single line of code, and have a circle show up on the screen. Add a few more lines of code, and the circle follows the mouse. Another line of code, and the circle changes color when the mouse is pressed. We call this sketching with code. You write one line, then add another, then another, and so on. The result is a program created one piece at a time.

Environment First, you'll need to get a code editor. A code editor is similar to a text editor (like Notepad or Notes), except it has special functionality for editing code instead of plain text. You can use any code editor you like; we recommend Atom and Brackets, both of which can be downloaded online.

Platform Dependency: This project is made in JavaScript solely and not using python which is the better option because it had to be integrated in a website. We had to keep it inside website because we needed to showcase it remotely because of pandemic situation.

p5.js helps in rendering the 2d graphics which is the best option for web based applications.

6 PROPOSED WORK

The Beginning: When the Sketch initially starts, the rockets have no Idea how to reach the target. The First thing the user will notice that the rockets are shooting in random directions and only none/few made it to it's destination.

The Evolution Begins: After few Generations passed, you may notice that the number of rockets reaching the target increasing. You can even visually see that the rockets become more Intelligent in determining their paths, learning from the mistakes of previous Generations. You may have added some Obstacles and the rockets start to avoid crashes and have a tendency of reaching the target.

The Smart World: At some point in time, when enough generations has passed (generally within 50-60 generations) you may see most of the Rockets in the Populations are reaching the target efficiently. Here, we have determined that when 30% of the population reaches the target, we declare it to be evolved for the given scenario (given combination of Obstacles and target position).

7 PROPOSED METHODOLOGY

Initialization: The population size depends on the nature of the problem, but typically contains several hundreds or thousands of possible solutions. Often, the initial population is generated randomly, allowing the entire range of possible solutions (the search space). Occasionally, the solutions may be "seeded" in areas where optimal solutions are likely to be found.

In our project this concept is demonstrated by the following code snippet -

```
for(leti=0;i<popsize;i++)          {
      this.rockets[i]=new Rocket();
  }

where popsize=populationsize
}
```

Selection: Selection is the component which guides the algorithm to the solution by preferring individuals with high fitness over low-fitted ones. It can be a deterministic operation, but in most implementations it has random components. One variant, which is very popular nowadays (we will give a theoretical explanation of its good properties later), is the following scheme, where the probability to choose a certain individual is proportional to its fitness.

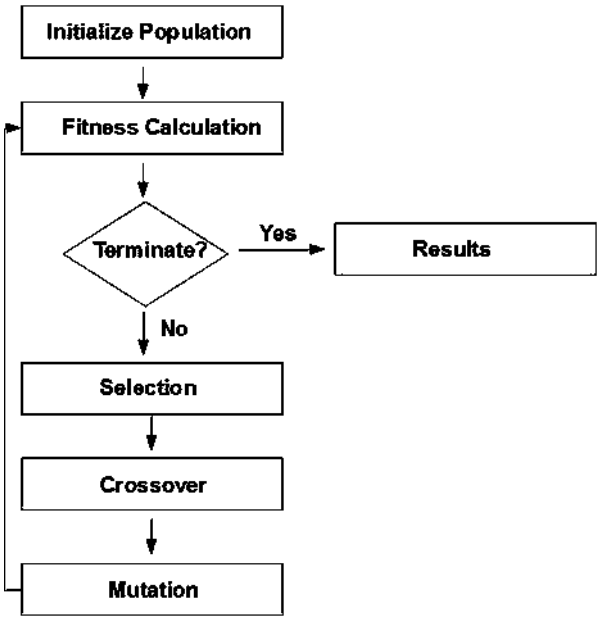


Figure 6: Flow Chart of the Methodology

In our project this concept is demonstrated by the following code snippet -

```
this.matingpool=[];
  for(leti=0;i<popsize;i++)          {
    var n = this.rockets[i].fitness * 100;
    for(letj=0;j<n;j++)          {
      this.matingpool.push(this.rockets[i]);
    }
  }
};

this.selection = function() {
  let newrockets=[];
  for(leti=0;i<
  this.rockets.length;i++) {
    letindex A=
    floor(random(this.matingpool.length));
    letindex B=
    floor(random(this.matingpool.length));
    letparent Adna=
    this.matingpool[indexA].dna;
    let parentBdna=
    this.matingpool[indexB].dna;
    let childdna=
    parent Adna.crossover(parentBdna);
    newrockets[i]=new Rocket(childdna);
  }
```

Genetic Operators: The simplest form of genetic algorithm involves three types of operators: selection, crossover (single point), and mutation.

Selection This operator selects chromosomes in the population

for reproduction. The fitter the chromosome, the more times it is likely to be selected to reproduce.

Crossover This operator randomly chooses a locus and exchanges the subsequences before and after that locus between two chromosomes to create two offspring. For example, the strings 10000100 and 11111111 could be crossed over after the third locus in each to produce the two offspring 10011111 and 11100100. The crossover operator roughly mimics biological recombination between two singlechromosome (haploid) organisms.

Mutation This operator randomly flips some of the bits in a chromosome. For example, the string 00000100 might be mutated in its second position to yield 01000100. Mutation can occur at each bit position in a string with some probability, usually very small (e.g., 0.001).

In our project this concept is demonstrated by the following code snippet -

Crossover code segment -

```
this.crossover = function(partner) {
  let newgenes = [];
  let mid =
  floor(random(this.genes.length));
  for(let i=0;i<
  this.genes.length;i++)i {
    f(i<mid)      {
      newgenes[i]= partner.genes[i];
    } else {
      newgenes[i] = this.genes[i];
    }
  }
  return new DNA( newgenes );
};
```

Mutation code segment -

```
this.mutation = function() {
  for(let i=0;i<
  this.genes.length;i++)i {
    f(random(1)
    <0.01)     // Mutation rate of 1%
      genes[i] = p5.Vector.random2D();
      this.genes[i].setMag(random(randomMag));
  }
};
```

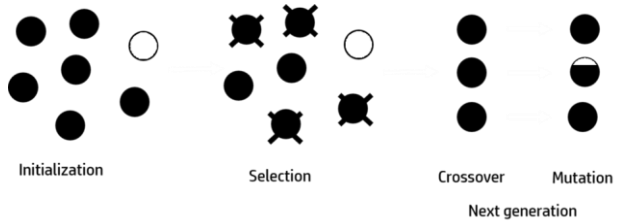


Figure 7: Initialisation-Selection-Next Generation

Termination: This generational process is repeated until a termination condition has been reached. Common terminating conditions are:

In our project this concept is demonstrated by the following code snippet -

```
if (count == lifespan) {
  for(let i=0;i<
  population.rockets.length;i++){
    if(population.rockets[i].completed)
    population.reached++;
    if(!population.rockets[i].completed)
    population.notreached++;
  }
  reached = population.reached;
  notreached=
  population.notreached;
  population.reached =
  population.notreached= 0;
  population.evaluate();
  population.selection();
  generation++;
  count=0;//RESET
}
```

8 DOCUMENTATION

8.1 Buttons and Sliders usage

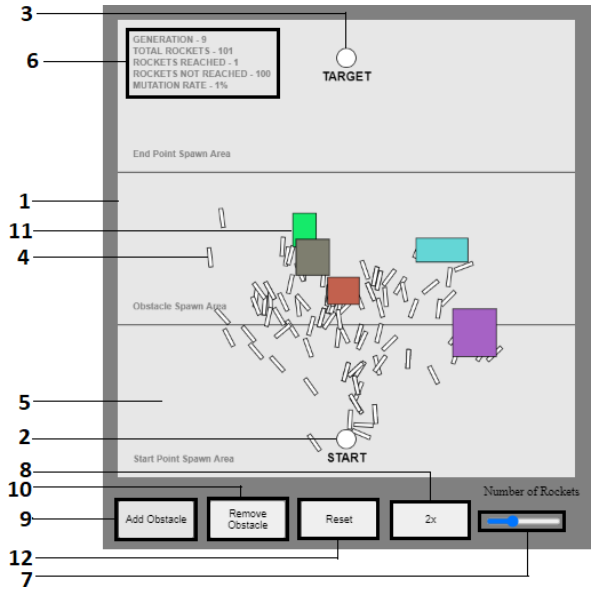


Figure 8: Reference of Attribute No.

All features and functionality of the application are marked and explained below in detail.

Attribute No.	Usage
1	This is the Obstacle Spawn Area. No obstacles will spawn out this perimeter to avoid errors and loopholes.
2	This is the Start Point. All Rockets will emerge from this point. **Note:** This will spawn in random position every time reset button is activated.
3	This is the end point. All Rockets will try to submerge on this point. **Note:** This will spawn in random position every time reset button is activated.
4	These are the Rockets. They will always stay inside of Canvas. Their driving mechanism is entirely controlled by Genetic Algorithm
5	This is the Start Point Spawn Area. The Start Point will not spawn out this perimeter to avoid errors and loopholes.
6	This is the main Info Panel. It shows crucial Info like **Generation:** The no. of generations passed **Total Rockets:** Total no. of Rockets are currently there in the canvas **Rockets Reached:** The no. rockets reaching the Target in the current generation **Rockets Not Reached:** The no. rockets not reaching the Target in the current generation **Mutation Rate:** The current mutation rate. **Note:**This is static and cannot be changed
7	This Slider changes the number of Rockets inside the canvas.**Note:** Changing this will reset generation count to zero, that is all genetic data gets reset
8	This button speeds up everything by 2x.**Note:**Once activated, cannot be disabled until Reset is used.
9	This button adds Obstacles in the Obstacle Spawn area.
10	This button removes Obstacles from the Obstacle Spawn area.
11	These are the Obstacles. No Rockets can move in the current generation once it crashes on one of these
12	This button Resets all data and positioning.

9 EXTENSION OF OUR PROJECT

Genetic algorithm is commonly used to generate high-quality solutions to optimization and search problems by relying on on biologically inspired operators such as mutation, crossover and selection.

Economics: GAs are also used to characterize various economic models like the cobweb model, game theory equilibrium resolution, asset pricing, etc. The automated design can help the financial sectors for their sophisticated trading system.

Neural Networks: Neural networks are one of the most beautiful programming paradigms ever invented. In the conventional approach to programming, we tell the computer what to do, breaking big problems up into many small, precisely defined tasks that the computer can easily perform. By contrast, in a neural network we don't tell the computer how to solve our problem. Instead, it learns from observational data, figuring out its own solution to the problem at hand.

Image Processing: Genetic algorithm is the type of Soft Computing method. The Genetic Algorithm (GA) is a model of machine learning which derives its behavior from a metaphor of the processes of evolution in nature. The aim is to enhance the quality of the image and to convert the image into segments to get more meaningful image and it will be easy to analyze the image using genetic algorithm. Genetic algorithm is the unbiased optimization technique. It is useful in image enhancement and segmentation.

Vehicle Routing Problems: The Vehicle Routing Problem with Time windows (VRPTW) is an extension of the capacity constrained Vehicle Routing Problem (VRP). It is a multi-objective problem and present a genetic algorithm solution using the Pareto ranking technique. It uses a direct interpretation of the VRPTW as a multi-objective problem, in which the two objective dimensions are number of vehicles and total cost (distance). This prevents the introduction of solution bias towards either of the problem dimensions. As a result, genetic algorithm returns a set of solutions that fairly consider both of these dimensions.

10 ACKNOWLEDGMENTS

Throughout the writing of this dissertation I have received a great deal of support and assistance.

I would first like to thank my supervisor, Dr. Saikat Maity, whose expertise was invaluable in the formulating of the research topic and methodology in particular.

I would like to acknowledge my peer for their wonderful collaboration. You supported me greatly and were always willing to help me.

In addition, I would like to thank my parents for their wise counsel and sympathetic ear. You are always there for me. Finally, there are my friends, who were of great support in deliberating over our problems and findings, as well as providing happy distraction to rest my mind outside of my research.

REFERENCES

(i) J. T. Alander, *An indexed bibliography of genetic algorithms*: Years 1957,Feb. 1994.

(ii) J. N. Amaral, K. Tumer and J. Ghosh, *"Designing genetic algorithms for the state assignment problem"*, IEEE Trans. Syst. Man Cybern., vol. 25, no. 4, Apr. 1995.

(iii) T. B äck, D.B. Fogel, Z. Michalewicz (Eds.), *Handbook of Evolutionary Computation*, IOP Publishing, 1997.

(iv) G. Bocharov, P. Klenerman, S. Ehl *Modelling the dynamics of LCMV infection in mice: II. Compartmental structure and immunopathology* J. Theor. Biol., 221 (2003), pp. 349-378

(v) Noraini Mohd Razali, John Geraghty *"A genetic algorithm performance with different selection strategies"*, Proceedings of the World Congress on Engineering Vol II, 2011

(vi) D. E. Goldberg, *Genetic Algorithm in Search, Optimization and Machine Learning* , Reading, MA: Addison-Wesley, 1989.

(vii) B. Doerr, A. Eremeev, C. Horoba, F. Neumann, and M. Theile, *"Evolutionary algorithms and dynamic programming,"* in Proceedings of the 11th annual conference on genetic and evolutionary computation, pp. 771–778, ACM, Québec, Canada, 2009.

(viii) M. Srinivas and L. M. Patnaik, *"Genetic algorithms: a survey,"* Computer, vol. 27, no. 6, pp. 17–26, 1994.

(ix) Yang, Q.: *Research on Problem of Item Bank Auto-Composing Test Paper Based on Genetic Algorithm.* Journal of University of Jinan (Science and Technology) 18(3), 228–231 (2004)

(x) Chen, X., Wang, H.: *A exam-paper generating algorithm based on improved genetic algorithm*. Journal of Harbin Institute of Technology 37(9), 1174–1176 (2005)

(xi) E. J. Anderson and M. C. Ferris, *A genetic algorithm for the assembly line balancing problem*, 1990.

(xii) P. J. Angeline, G. M. Saunders and J. B. Pollack, *"An evolutionary algorithm that constructs recurrent neural networks"*, IEEE Trans. Neural Networks, vol. 5, pp. 54-65, Jan. 1994

(xiii) *"The evolution of strategies in the iterated prisoner's dilemma"*, Genetic Algorithms and Stimulated Annealing, pp. 32-41, 1987.

(xiv) textit"Adaptive selection methods for genetic algorithms", Proc. 1st Int. Conf. Genetic Algorithms., pp. 101-111, 1985.

(xv) textit"Reducing bias and inefficiency in the selection algorithm", Proc. 2nd Int. Conf. Genetic Algorithms, pp. 14-21, 1987.

(xvi) L. Davis *Handbook of Genetic Algorithms* Van Nostrand Reinhold, New York (1991)

(xvii) D.E. Goldberg *Genetic Algorithms in Search, Optimization and Machine Learning* Addison-Wesley, Reading, MA (1989)

(xviii) Coello, Carlos. *"An updated survey of GA-based multiobjective optimization techniques."* ACM Computing Surveys, vol.32, no.2, p.109-143 (June 2000).

(xix) Fonseca, Carlos and Peter Fleming. *"An overview of evolutionary algorithms in multiobjective optimization."* Evolutionary Computation, vol.3, no.1, p.1-16 (1995).

(xx) K. Krishna and M. N. Murty, "Genetic K-Means Algorithm", IEEE Transaction on Systems, Man, and Cybernetics—Part B: CYBERNETICS, Vol. 29, No. 3, June 1999.

(xxi) Radwan A., Latef B., Ali A., and Sadek O., *"Using Genetic Algorithm to Improve Information Retrieval Systems"*, World Academy of Science and Engineering Technology, 17, Issue-2, Page No (6-13), 2006.